

Café Control

Sistema de compra y venta de café
Manual técnico para principiantes

Programado con HTML, CSS, JavaScript, PHP y MySQL
Versión 1.0 · Documentación educativa

Contenido: Explicación de cada módulo y de cada parte del código, paso a paso, pensado para personas que recién comienzan.

Índice

1. ¿Qué es este sistema y cómo está hecho?
2. Las tecnologías que se usan (explicadas fácil)
3. Estructura de carpetas y archivos
4. ¿Cómo se ejecuta? El viaje de una petición
5. Piezas base: configuración, conexión y utilidades
6. La plantilla visual: header, sidebar y CSS
7. Módulo de Acceso (login) y seguridad
8. Módulo 1 · Registro de Proveedores
9. Módulo 2 · Registro de Clientes
10. Módulo 3a · Gestión de Compra
11. Módulo 3b · Gestión de Venta
12. Módulo 4 · Reportes
13. Usuarios y Configuración (tasa de cambio)
14. La base de datos, tabla por tabla
15. Glosario de conceptos para principiantes
16. Cómo instalar y ejecutar el sistema

1. ¿Qué es este sistema y cómo está hecho?

Café Control es una aplicación web que permite administrar la compra y venta de café en tres versiones: **Suave**, **Normal** y **Fuerte**.

Reglas del negocio que resuelve

- El café se **compra por kilogramo** a los proveedores, **al contado**, y los precios se manejan en **dólares (USD)**.
- El café se **vende en bolívares (Bs)** a los clientes (acreedores), con **financiamiento de hasta una semana (7 días)**.
- El sistema calcula las **ganancias** convirtiendo el costo (en dólares) a bolívares usando una **tasa de cambio editable**.

Los cuatro módulos

Módulo	Qué hace	Archivo principal
1. Proveedores	Agregar, modificar solo el contacto y desactivar negocios que nos venden café.	`proveedores.php`
2. Clientes	Agregar, modificar solo el contacto y desactivar negocios a los que les vendemos café.	`clientes.php`
3. Compra / Venta	Registrar compras (USD, contado) y ventas (Bs, crédito ≤ 7 días) con tipos de café, kg y montos.	`compras.php` y `ventas.php`
4. Reportes	Ganancias diarias/semanales, cuentas por cobrar, inventario y compras por proveedor.	`reportes.php`

> **Además**, el sistema pide **iniciar sesión** (sección de acceso), valida todos los datos que se escriben y muestra un **panel de inicio** con indicadores.

2. Las tecnologías que se usan (explicadas fácil)

Tecnología	Para qué sirve aquí	Analogía
HTML	Es el "esqueleto": define los textos, campos, botones y tablas de cada página.	Los ladrillos y paredes de una casa.
CSS	Es la "decoración": colores, tamaños, espacios y el diseño con tonos de café.	La pintura y los muebles.
JavaScript (JS)	Da vida en el navegador: valida al instante, suma los montos, abre ventanas (modales).	La electricidad y los interruptores.
PHP	El "cerebro" en el servidor: recibe los datos, valida, guarda y consulta la base de datos.	El gerente que trabaja por detrás.
MySQL	La "memoria": guarda de forma permanente proveedores, clientes, compras y ventas.	Un archivador gigante.
Apache	El "portero" que entrega las páginas al navegador (viene dentro de XAMPP).	El mostrador de atención.

Conceptos extra que aparecen en el código

- **PDO:** la "tubería segura" de PHP hacia MySQL. Permite hacer consultas parametrizadas (combinadas con *prepared statements*) que evitan que alguien inyecte código malicioso. Es como enviar un formulario por mensajería sellada en vez de dictarlo en voz alta.
- **Sesión (session):** una memoria temporal del servidor que recuerda quién inició sesión mientras navega. Es la pulsera que te dan en un evento: con ella puedes entrar a todas las salas.
- **CSRF token:** una clave secreta que viaja con cada formulario para asegurar que el envío viene de nuestra propia página y no de un sitio externo malicioso.
- **Validación doble:** se revisa JavaScript en el navegador (rápido, para el usuario) y PHP en el servidor (definitivo, por seguridad).

3. Estructura de carpetas y archivos

```
sistema_cafe/
├── config/
│   ├── config.php        // Datos de conexión y constantes
│   └── database.php      // Crea la conexión PDO a MySQL
├── includes/
│   ├── functions.php     // Funciones de ayuda (validar, formatear...)
│   ├── auth.php          // Sesiones, login y permisos
│   ├── header.php        // Barra superior + menú lateral (parte de arriba)
│   └── footer.php        // Cierre del HTML + carga de JavaScript
├── assets/
│   ├── css/estilo.css    // Todo el diseño visual
│   └── js/app.js         // Validación y efectos en el navegador
├── database/schema.sql   // Estructura de la base de datos
├── index.php             // Pantalla de acceso (login)
├── logout.php            // Cerrar sesión
├── instalar.php          // Crea la base de datos y el usuario admin
├── dashboard.php         // Panel de inicio con indicadores
├── proveedores.php       // Módulo 1
├── clientes.php          // Módulo 2
├── compras.php           // Módulo 3a
├── ventas.php            // Módulo 3b
├── reportes.php          // Módulo 4
├── usuarios.php          // Gestión de usuarios (admin)
└── configuracion.php     // Tasa de cambio (admin)
```

> **¿Por qué separar en carpetas?** Para no repetir código. Por ejemplo, `header.php` (el menú) se escribe **una sola vez** y se reutiliza en todas las páginas. A esto se le llama *reutilización de código*.

4. ¿Cómo se ejecuta? El viaje de una petición

Cuando el usuario escribe `http://localhost/sistema_cafe/ventas.php` en el navegador, ocurre esto:

1. **Apache** recibe la petición y se la pasa a **PHP**.
2. PHP ejecuta `ventas.php` de arriba hacia abajo. Primero revisa si hay sesión iniciada (si no, te manda al login).
3. PHP se conecta a **MySQL** con PDO y pide los datos (clientes, tipos de café, últimas ventas).
4. PHP "arma" el HTML final mezclando el código con los datos (esto es *renderizar*) y se lo devuelve a Apache.
5. Apache entrega ese HTML al navegador, que lo muestra. El **CSS** le da estilo y el **JavaScript** añade la interactividad.

> **Idea clave:** PHP se ejecuta en el *servidor* (no lo ve el usuario). JavaScript, HTML y CSS se ejecutan en el *navegador*. Por eso PHP es para la lógica y los datos, y JS para lo visual y lo inmediato.

5. Piezas base: configuración, conexión y utilidades

5.1 config/config.php — los datos que casi nunca cambian

```php
<?php
define('APP_NAME', 'Café Control');   // Nombre del sistema
define('DB_HOST', '127.0.0.1');        // Dirección del servidor MySQL
define('DB_NAME', 'cafe_db');          // Nombre de la base de datos
define('DB_USER', 'root');             // Usuario de MySQL (XAMPP)
define('DB_PASS', '');                 // Contraseña (vacía en XAMPP)
date_default_timezone_set('America/Caracas');
```

- `define('NOMBRE', valor)` crea una **constante**: un valor con nombre que no cambia durante la ejecución. Se usa con solo escribir `DB_HOST`.
- Guardar aquí los datos de conexión permite cambiarlos en **un solo lugar** si el sistema se muda a otro servidor.

5.2 config/database.php — la conexión (PDO)

```php
function db(): PDO {
    static $pdo = null;
    if ($pdo === null) {
        $dsn = 'mysql:host=' . DB_HOST . ';dbname=' . DB_NAME;
        try {
            $pdo = new PDO($dsn, DB_USER, DB_PASS, [
                PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
                PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
            ]);
        } catch (PDOException $e) {
            die('Error de conexión: ' . $e->getMessage());
        }
    }
    return $pdo;
}
```

- `static $pdo = null;` crea una variable que **se recuerda entre llamadas**. Así la conexión se abre una sola vez y se reutiliza (más rápido).
- `$dsn` es la "dirección" que dice a qué motor y base conectarse.
- `try / catch` significa: "intenta esto; si falla, haz esto otro". Si MySQL está apagado, mostramos un mensaje en vez de una pantalla en blanco.
- Luego, en cualquier página, basta con escribir `db()` para obtener la conexión.

5.3 includes/functions.php — utilidades de uso diario

Aquí viven "atajos" que usamos en todas las páginas. Los más importantes:

```php
// Escapa texto para mostrarlo seguro en HTML (evita inyección de etiquetas)
function e($v): string {
    return htmlspecialchars((string)$v, ENT_QUOTES, 'UTF-8');
}

// Muestra un monto en bolívares: "Bs 1.500,00"
function bs($n): string {
    return 'Bs ' . number_format((float)$n, 2, ',', '.');
}

// Redirige a otra página y detiene la ejecución
function redirect(string $url): void {
    header('Location: ' . $url);
    exit;
}
```

- `e($texto)` convierte caracteres peligrosos (`<`, `>`) en texto inofensivo. **Siempre** se usa al imprimir datos del usuario, por ejemplo `<?= e($cliente['nombre']) ?>`.
- `number_format` da formato a los números con separadores de miles y decimales.
- `redirect` evita que, al recargar, se vuelva a enviar un formulario ya procesado.

Validadores de datos

El sistema revisa RIF, cédula, teléfono y correo antes de guardar. Ejemplo:

```php
function rif_valido(string $rif): bool {
    return (bool) preg_match('/^[VEJPGvejpg]-?\d{7,9}-?\d?$/', trim($rif));
}
function email_valido(string $email): bool {
    return (bool) filter_var(trim($email), FILTER_VALIDATE_EMAIL);
}
```

- `preg_match` comprueba si el texto cumple un **patrón** (por ejemplo, que el RIF empiece con J, V, E, G o P seguido de números).
- `filter_var(..., FILTER_VALIDATE_EMAIL)` es la función estándar de PHP para saber si un correo es válido.

6. La plantilla visual: header, sidebar y CSS

6.1 includes/header.php — barra superior y menú

Este archivo se incluye al inicio de **todas** las páginas con una línea ISP: `<?php require_once __DIR__ . '/includes/header.php'; ?>`

```
// 1) Obliga a tener sesión iniciada (si no, al login).
require_once __DIR__ . '/auth.php';
require_login();

$user = current_user();        // Datos del usuario que entró
$tasa = tasa_cambio();         // Tasa Bs/$ actual, leída de la BD

$menu = [
  ['id' => 'proveedores', 'titulo' => 'Proveedores', 'url' => 'proveedores.php', 'admin' =>
false],
  // ... una entrada por cada módulo
];
```

- `$menu` es un **array**: una lista ordenada de opciones. Cada opción es a su vez un pequeño grupo con `id`, `titulo`, `url` y si es solo para `admin`.
- Más abajo, un bucle `foreach` recorre el menú y crea los enlaces automáticamente. Las opciones con `'admin' => true` se saltan si el usuario no es administrador (`if ($m['admin'] && !is_admin()) continue;`).

```
foreach ($menu as $m):
    if ($m['admin'] && !is_admin()) continue; ?>
    <a class="nav-link <?= $PAGE === $m['id'] ? 'active' : '' ?>"
       href="<?= e($m['url']) ?>"><?= e($m['titulo']) ?></a>
<?php endforeach; ?>
```

`$PAGE` guarda el identificador de la página actual; comparándolo con `$m['id']` se marca como "activo" (entonces el CSS lo pinta resaltado) la opción del menú donde estamos.

6.2 includes/footer.php — cierre de página

```
    </main>
</div>
<script src="assets/js/app.js"></script>
</body>
</html>
```

Cierra las etiquetas abiertas en `header.php` y carga el archivo JavaScript al final (para que la página se muestre antes de cargar los scripts).

6.3 assets/css/estilo.css — el diseño

Se usan **variables CSS** para mantener una paleta coherente de café:

```
:root {
    --cafe-800: #3d2418;   /* marrón oscuro */
    --crema:    #f7efe4;   /* fondo claro   */
    --radio:    12px;      /* bordes redondeados */
}
.topbar { background: var(--cafe-800); }
```

Con `--nombre` se define un valor y con `var(--nombre)` se usa. Si mañana se quiere otra tonalidad, se cambia en un solo sitio.

6.4 assets/js/app.js — validación en el navegador

Sus tres tareas:

1. **Abrir/cerrar el menú** en celulares (`sidebar.classList.toggle('open')`).
2. **Validar campos al escribir** usando el atributo `data-validate` de cada input.
3. **Abrir y cerrar ventanas modales** (las de "Agregar proveedor", etc.).

```
const regex = {
    rif:     /^[VEJPGvejpg]-?\d{7,9}-?\d?$/,
    email:   /^[^\s@]+@[^\s@]+\.[^\s@]+$/,
    positivo:/^\d+(\.\d+)?$/
};

function validarCampo(input) {
    const val  = input.value.trim();
    const tipo = input.dataset.validate;   // lee data-validate
    let ok = true;
    if (input.required && val === '') ok = false;
    if (ok && tipo === 'email') ok = regex.email.test(val);
    // ... otras comprobaciones ...
    input.closest('.field').classList.toggle('invalid', !ok);
    return ok;
}
```

La clase `invalid` enciende con CSS el borde rojo y el mensaje de error. Así el usuario ve el problema **mientras escribe**, sin esperar al servidor.

7. Módulo de Acceso (login) y seguridad

7.1 index.php — la pantalla de acceso

```php
<?php
require_once __DIR__ . '/includes/auth.php';

if (current_user()) { redirect('dashboard.php'); }  // Si ya entró, al panel

if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    verify_csrf();                                  // Revisa el token de seguridad
    $usuario = trim($_POST['usuario'] ?? '');
    $clave   = $_POST['clave'] ?? '';
    if ($usuario === '' || $clave === '') {
        $error = 'Debes completar usuario y contraseña.';
    } else {
        [$ok, $msg] = attempt_login($usuario, $clave);  // Intenta validar
        if ($ok) { flash($msg); redirect('dashboard.php'); }
        $error = $msg;
    }
}
?>
```

- `$_SERVER['REQUEST_METHOD'] === 'POST'` pregunta: "¿el usuario envió el formulario?" Si la página se abrió normalmente (GET), muestra el formulario; si lo envió (POST), procesa el login.
- `[$ok, $msg] = attempt_login(...)` recibe **dos valores a la vez** (éxito y mensaje). Se llama *desestructuración*.

```php
<form class="login-card" method="post" data-validar novalidate>
    <?= csrf_field() ?>
    <input type="text" name="usuario" required>
    <input type="password" name="clave" required>
    <button type="submit">Ingresar</button>
</form>
```

`csrf_field()` imprime un campo oculto con la clave de seguridad. `method="post"` hace que los datos viajen en el "cuerpo" de la petición (no en la dirección).

7.2 includes/auth.php — sesiones y permisos

```php
if (session_status() === PHP_SESSION_NONE) { session_start(); }

function attempt_login(string $usuario, string $clave): array {
    $st = db()->prepare('SELECT * FROM usuarios WHERE usuario = ? LIMIT 1');
    $st->execute([trim($usuario)]);
    $u = $st->fetch();

    if (!$u || !password_verify($clave, $u['password_hash'])) {
```

```
        return [false, 'Usuario o contraseña incorrectos.'];
    }
    $_SESSION['user'] = [ 'id' => $u['id'], 'nombre' => $u['nombre'], 'rol' => $u['rol'] ];
    return [true, 'Bienvenido'];
}
```

- `session_start()` activa la memoria de sesión del servidor.
- La consulta usa `?` como **marcador** y luego `execute([...])` entrega el valor. Esto es un *prepared statement*: el usuario nunca se "mezcla" con el código SQL. Es la defensa principal contra inyección SQL.
- `password_verify` compara la contraseña escrita con la **versión cifrada** guardada. Nunca guardamos la clave real, solo su cifrado (hash).
- Al entrar, guardamos en la sesión quién es (`$_SESSION['user']`).

Proteger las páginas

```
function require_login(): void {
    if (!current_user()) redirect('index.php?msg=login');
}
function require_admin(): void {
    require_login();
    if (!is_admin()) { flash('Sin permisos', 'error'); redirect('dashboard.php'); }
}
```

Cada página llama a `require_login()` (o `require_admin()`) al principio. Si el usuario no tiene sesión, se le redirige automáticamente al acceso.

8. Módulo 1 · Registro de Proveedores

Archivo: `proveedores.php`. Es el patrón que también usa Clientes, así que entenderlo bien sirve para ambos.

8.1 Estructura general del archivo

1. Cargar el encabezado (que exige sesión).
2. Si llegó un formulario (**POST**), procesarlo según la acción: crear, editar contacto o cambiar estado.
3. Consultar la lista de proveedores.
4. Dibujar la tabla y los formularios en ventanas modales.

```
$PAGE = 'proveedores';
require_once __DIR__ . '/includes/header.php';
$pdo = db();
$errores = [];
```

8.2 Agregar un proveedor (CREATE)

```
if ($_SERVER['REQUEST_METHOD'] === 'POST') {
    verify_csrf();
    $accion = $_POST['accion'] ?? '';

    if ($accion === 'crear') {
        // 1) Recibir y limpiar los datos
        $nombre = trim($_POST['nombre_negocio'] ?? '');
        $rif    = strtoupper(trim($_POST['rif'] ?? ''));
        // ... contacto_nombre, cedula, telefono, email ...

        // 2) Validar
        if ($nombre === '')     $errores[] = 'El nombre es obligatorio.';
        if (!rif_valido($rif))  $errores[] = 'RIF no válido.';

        // 3) Si no hay errores, guardar
        if (!$errores) {
            $st = $pdo->prepare('INSERT INTO proveedores
                (nombre_negocio, rif, contacto_nombre, ...) VALUES (?, ?, ?, ...)');
            $st->execute([$nombre, $rif, $cn, $cc, $ct, $ce]);
            flash('Proveedor registrado.');
            redirect('proveedores.php');
        }
    }
}
```

- `trim()` quita espacios de los extremos; `strtoupper()` pasa el RIF a mayúsculas.

- `$errores[] = '...'` agrega un mensaje a la lista de errores. Solo si la lista queda vacía se guarda.
- `INSERT INTO proveedores` es la orden SQL que agrega una fila nueva.
- `flash(...)` deja un mensaje que aparecerá una sola vez en la próxima página (el clásico "Guardado correctamente").

> **RIF repetido:** la tabla tiene el RIF como `UNIQUE` (no se puede repetir). Si se intenta, MySQL lanza una excepción `23000` que el código captura para mostrar "Ya existe un proveedor con ese RIF".

8.3 Modificar SOLO el contacto (UPDATE)

El requisito dice que del negocio solo se pueden editar los datos de contacto. Por eso la consulta de actualización **no toca** `nombre_negocio` ni `rif`:

```
if ($accion === 'editar_contacto') {
    if (!$errores) {
        $st = $pdo->prepare('UPDATE proveedores
            SET contacto_nombre = ?, contacto_cedula = ?,
                contacto_telefono = ?, contacto_email = ?
            WHERE id = ?');
        $st->execute([$cn, $cc, $ct, $ce, $id]);
        redirect('proveedores.php');
    }
}
```

En el formulario, el nombre del negocio y el RIF se muestran como `readonly` (solo lectura), así el usuario los ve pero no los puede cambiar.

8.4 Desactivar (baja lógica)

```
if ($accion === 'toggle') {
    $st = $pdo->prepare('UPDATE proveedores SET activo = 1 - activo WHERE id = ?');
    $st->execute([$id]);
}
```

No se borra el proveedor: solo se cambia su columna `activo` entre 1 y 0. `1 - activo` invierte el valor (si era 1 pasa a 0 y viceversa). Esto se llama **baja lógica** y conserva el historial de compras.

8.5 La tabla y las ventanas modales

```
<?php foreach ($proveedores as $p): ?>
  <tr class="<?= $p['activo'] ? '' : 'inactivo' ?>">
    <td><strong><?= e($p['nombre_negocio']) ?></strong></td>
    <td><?= $p['activo'] ? 'Activo' : 'Inactivo' ?></td>
```

```
    </tr>
<?php endforeach; ?>
```

El operador `condición ? valor_si : valor_no` es un **"si... entonces... si no"** corto. Aquí decide qué texto y qué color mostrar según el estado.

9. Módulo 2 · Registro de Clientes

Archivo: `clientes.php`. Es prácticamente **idéntico** a Proveedores: mismos campos (nombre del negocio, RIF y datos de contacto), misma validación, misma baja lógica. La única diferencia es que guarda en la tabla `clientes` y usa los rótulos "cliente/acreedor".

> **Reutilizar el patrón:** al entender Proveedores ya se entiende Clientes. Solo cambian el nombre de la tabla en las consultas SQL y algunos textos.

```
// En clientes.php cambia solo la tabla:
INSERT INTO clientes (...) VALUES (...);
UPDATE clientes SET activo = 1 - activo WHERE id = ?;
```

10. Módulo 3a · Gestión de Compra

Archivo: `compras.php`. Registra una compra a un proveedor: se elige el proveedor, la fecha y varias líneas de "tipo de café + kg + precio en dólares". El total es en USD (pago de contado).

10.1 Procesar las líneas (varios tipos a la vez)

```
$tipos      = $_POST['tipo_cafe_id'] ?? [];
$cantidades = $_POST['cantidad_kg'] ?? [];
$precios    = $_POST['precio_usd_kg'] ?? [];

$lineas = []; $totalUsd = 0;
foreach ($tipos as $i => $tipoId) {
    $cant   = (float)$cantidades[$i];
    $precio = (float)$precios[$i];
    if ($tipoId <= 0 || $cant <= 0 || $precio <= 0) continue;  // línea vacía o inválida
    $monto = round($cant * $precio, 2);
    $lineas[] = ['tipo' => $tipoId, 'cant' => $cant, 'precio' => $precio, 'monto' => $monto];
    $totalUsd += $monto;
}
```

- El formulario envía tres listas paralelas: `tipo_cafe_id[]`, `cantidad_kg[]` y `precio_usd_kg[]`. Los corchetes `[]` en el `name` hacen que PHP las reciba como **arreglos**.
- El `foreach` con `$i => $tipoId` recorre cada línea; el índice `$i` sirve para leer su cantidad y precio.
- `continue` salta a la siguiente línea cuando los datos no sirven.

10.2 Guardar cabecera + detalle (con transacción)

```
if (!$errores) {
    try {
        $pdo->beginTransaction();                // Todo o nada
        $folio = generar_folio('C', 'compras');
        $st = $pdo->prepare('INSERT INTO compras (folio, proveedor_id, fecha, total_usd,
usuario_id)
                             VALUES (?, ?, ?, ?, ?)');
        $st->execute([$folio, $proveedor_id, $fecha, $totalUsd, $user['id']]);
        $compraId = $pdo->lastInsertId();          // id de la compra recién creada

        $std = $pdo->prepare('INSERT INTO compra_detalle
            (compra_id, tipo_cafe_id, cantidad_kg, precio_usd_kg, monto_usd) VALUES (?, ?, ?,
?, ?)');
        foreach ($lineas as $l) {
            $std->execute([$compraId, $l['tipo'], $l['cant'], $l['precio'], $l['monto']]);
        }
        $pdo->commit();                          // Confirmar y grabar de forma segura
    } catch (Throwable $ex) {
        $pdo->rollBack();                        // Si algo falla, deshacer todo
```

```
    }
}
```

- **Transacción** (`beginTransaction` / `commit` / `rollBack`): agrupa varias órdenes SQL como una sola operación. Si un solo detalle falla, se deshace toda la compra. Así nunca queda una cabecera sin sus líneas.
- `beginTransaction` = "empecemos a anotar en borrador"; `commit` = "listo, imprímelo definitivo"; `rollBack` = "tira todo a la basura".

10.3 El JavaScript que suma en vivo

```
function recalcular() {
    let total = 0;
    cont.querySelectorAll('.linea-input').forEach(row => {
        const cant   = parseFloat(row.querySelector('.inp-cant').value) || 0;
        const precio = parseFloat(row.querySelector('.inp-precio').value) || 0;
        total += cant * precio;
    });
    document.getElementById('totalGeneral').textContent = '$ ' + total.toFixed(2);
}
cont.addEventListener('input', recalcular);  // Cada tecla recalcula
```

`querySelectorAll` busca **todas** las líneas; `forEach` las recorre; se multiplica y se suma. El resultado se coloca en la caja "Total compra" sin recargar la página.

11. Módulo 3b · Gestión de Venta

Archivo: `ventas.php`. Similar a Compras, pero la venta es en **bolívares**, con **financiamiento ≤ 7 días** y guarda la **tasa de cambio** del momento para poder calcular la ganancia luego.

11.1 Validar el financiamiento (máximo 7 días)

```
$dias = (strtotime($vencimiento) - strtotime($fecha)) / 86400;   // segundos -> días
if ($dias < 0)  $errores[] = 'El vencimiento no puede ser anterior.';
if ($dias > 7)  $errores[] = 'No puede superar una semana.';
```

`strtotime` convierte una fecha en un número de segundos. La diferencia entre dos fechas se divide entre 86400 (los segundos que tiene un día) para obtener los días. Este control asegura la regla de negocio.

En el navegador, `ajustarFechas()` además fija automáticamente el máximo del calendario a 7 días después:

```
const max = new Date(f); max.setDate(max.getDate() + 7);
venc.max = iso(max);   // El selector de fecha no dejará elegir más de 7 días
```

11.2 Guardar el costo de cada línea (para las ganancias)

```
// Costo promedio de un tipo de café según TODAS las compras
function costo_promedio_usd(int $tipoId): float {
    $st = db()->prepare('SELECT COALESCE(SUM(monto_usd)/NULLIF(SUM(cantidad_kg),0),0)
                         FROM compra_detalle WHERE tipo_cafe_id = ?');
    $st->execute([$tipoId]);
    return (float) $st->fetchColumn();
}
```

Al vender, se guarda una "foto" de ese costo en la columna `costo_usd_kg` de la venta. Así, aunque después cambien los precios de compra, la ganancia histórica queda congelada y correcta. `NULLIF(...,0)` evita dividir entre cero.

11.3 Cobrar y anular

```
if ($accion === 'pagar') {
    $pdo->prepare("UPDATE ventas SET estado_pago='pagada', fecha_pago=CURDATE()
                   WHERE id = ? AND estado_pago='pendiente'")->execute([$id]);
}
if ($accion === 'anular') {
    $pdo->prepare("UPDATE ventas SET estado_pago='anulada' WHERE id = ?")->execute([$id]);
}
```

Una venta nace `pendiente` (cuenta por cobrar). Al pulsar "Cobrar" pasa a `pagada`; si se anula, queda como `anulada` y deja de contar en los reportes.

12. Módulo 4 · Reportes

Archivo: `reportes.php`. Usa un parámetro de la dirección (`?r=...`) para mostrar un reporte u otro, sin crear archivos separados. Esta técnica se llama **enrutado simple**.

```
$r = $_GET['r'] ?? 'ganancias';   // qué reporte mostrar
if (!isset($tabs[$r])) $r = 'ganancias';   // seguridad: valor permitido
```

12.1 Reporte de ganancias (por día, por semana y por negocio)

La ganancia de una venta se calcula así: **ganancia = monto vendido (Bs) − cantidad × costo_usd × tasa_cambio**.

```
SELECT v.fecha AS periodo,
       SUM(vd.monto_bs) AS ventas,
       SUM(vd.cantidad_kg * vd.costo_usd_kg * v.tasa_cambio) AS costo
FROM venta_detalle vd
JOIN ventas v   ON v.id = vd.venta_id
JOIN clientes c ON c.id = v.cliente_id
WHERE v.estado_pago <> 'anulada' AND v.fecha BETWEEN ? AND ?
GROUP BY v.fecha
ORDER BY v.fecha
```

- `JOIN` "pega" dos tablas por su relación (cada detalle conoce su venta y su cliente).
- `SUM(...)` suma todos los renglones del grupo; `GROUP BY v.fecha` agrupa por día.
- `BETWEEN ? AND ?` filtra por el rango de fechas que escribe el usuario.
- Cambiando `GROUP BY v.fecha` por `YEARWEEK(v.fecha,1)` se obtiene el mismo cálculo **semanal**, y cambiándolo por `c.nombre_negocio` se obtiene **por negocio**. Con eso se cumplen los tres enfoques pedidos.

En PHP, tras obtener ventas y costo, la ganancia se saca con una resta:

```
$g = (float)$f['ventas'] - (float)$f['costo'];
```

12.2 Cuentas por cobrar semanal

Son las ventas con `estado_pago = 'pendiente'`. Se agrupan por la semana de vencimiento (`YEARWEEK(fecha_vencimiento,1)`) y se resaltan las vencidas comparando con `CURDATE()` (la fecha de hoy).

```
SELECT YEARWEEK(fecha_vencimiento,1) AS semana,
       COUNT(*) AS cantidad, SUM(total_bs) AS total
FROM ventas
WHERE estado_pago = 'pendiente'
GROUP BY YEARWEEK(fecha_vencimiento,1)
```

12.3 Reportes adicionales (3 y 4)

- **Inventario:** por cada tipo de café calcula *comprado − vendido = disponible*, su costo promedio y el valor del inventario en dólares.
- **Compras por proveedor:** suma cuánto se le ha comprado a cada proveedor (en USD y en kg) dentro del rango de fechas.

12.4 Imprimir

El botón "Imprimir" llama a `window.print()`. El CSS incluye una sección `@media print` que oculta el menú y la barra superior al imprimir, para que el papel salga limpio.

13. Usuarios y Configuración (tasa de cambio)

13.1 usuarios.php — solo para administradores

Empieza con `require_admin()`, así que un vendedor no puede entrar. Permite crear usuarios, cambiar contraseñas y activar/desactivar. La contraseña **nunca** se guarda tal cual:

```
$st = $pdo->prepare('INSERT INTO usuarios (nombre, usuario, password_hash, rol) VALUES (?, ?, ?, ?)');
$st->execute([$nombre, $usuario, password_hash($clave, PASSWORD_DEFAULT), $rol]);
```

`password_hash` transforma la clave en un texto cifrado imposible de revertir.

13.2 configuracion.php — la tasa Bs/$ editable

La tasa se guarda en la tabla `configuracion` como un par clave/valor. Al editarla:

```
function config_set(string $clave, $valor): void {
    $st = db()->prepare('INSERT INTO configuracion (clave, valor) VALUES (?, ?)
                         ON DUPLICATE KEY UPDATE valor = VALUES(valor)');
    $st->execute([$clave, (string)$valor]);
}
```

"INSERT ... ON DUPLICATE KEY UPDATE" significa: *si la clave no existe, créala; si existe, actualízala*. Es como un "guardar o sobrescribir" en una sola orden.

14. La base de datos, tabla por tabla

Se usa la base `cafe_db`. Cada "tabla" es como una hoja de cálculo con columnas fijas.

Tabla	Qué guarda
`usuarios`	Quién puede entrar: nombre, usuario, clave cifrada, rol y si está activo.
`configuracion`	Parámetros como `tasa_cambio` (Bs por dólar).
`tipos_cafe`	Los tres tipos: Suave, Normal y Fuerte.
`proveedores`	Negocios que nos venden café (con RIF, contacto y estado).
`clientes`	Negocios a los que vendemos café (con RIF, contacto y estado).
`compras`	Cabecera de cada compra: folio, proveedor, fecha y total en USD.
`compra_detalle`	Cada línea de la compra: tipo, kg, precio $/kg y monto.
`ventas`	Cabecera de cada venta: folio, cliente, fechas, tasa, total Bs y estado de pago.
`venta_detalle`	Cada línea de la venta: tipo, kg, precio Bs/kg, costo y monto.

14.1 Ejemplo de definición de tabla

```
CREATE TABLE IF NOT EXISTS productos_o_ventas (
  id            INT AUTO_INCREMENT PRIMARY KEY,    -- número único que se asigna solo
  nombre        VARCHAR(150) NOT NULL,             -- texto obligatorio
  precio        DECIMAL(12,2) NOT NULL,            -- número con 2 decimales
  creado_en     DATETIME DEFAULT CURRENT_TIMESTAMP, -- fecha automática
  CONSTRAINT fk_xxx FOREIGN KEY (otro_id) REFERENCES otra_tabla(id) -- relación
) ENGINE=InnoDB;
```

- `PRIMARY KEY`: identificador único de cada fila (nunca se repite).
- `AUTO_INCREMENT`: MySQL asigna 1, 2, 3... automáticamente.
- `VARCHAR(n)`: texto de hasta n caracteres; `DECIMAL(12,2)`: número con 2 decimales (para dinero).
- `FOREIGN KEY`: una relación. Por ejemplo, cada compra apunta a un proveedor válido; MySQL impide guardar un proveedor inexistente.

> El archivo `instalar.php` crea todas estas tablas y añade el usuario `admin` y los tres tipos de café la primera vez.

15. Glosario de conceptos para principiantes

Término	Qué significa en este sistema
Variable	Una caja con nombre donde se guarda un dato (ej: `$nombre = 'Café'`).
Array	Una lista de varios valores bajo un solo nombre (ej: el menú del sistema).
Función	Un bloque de código con nombre que se puede reutilizar (ej: `bs()`, `db()`).
GET / POST	GET = pedir una página; POST = enviar datos de un formulario.
Prepared statement	Consulta SQL con `?` que separa el código de los datos: evita la inyección SQL.
Transacción	Grupo de operaciones "todo o nada" (ej: guardar una compra con todos sus detalles).
Sesión	Memoria del servidor que recuerda al usuario logueado.
Hash	Contraseña cifrada irreversible; se usa con `password_hash` y `password_verify`.
CSRF token	Clave secreta en cada formulario para confirmar que el envío es legítimo.
JOIN	Combinar dos tablas por su relación (venta + cliente).
Renderizar	Que PHP arme el HTML final y lo envíe al navegador.

16. Cómo instalar y ejecutar el sistema

1. Instala **XAMPP** y abre el *Control Panel*.
2. Pulsa **Start** en **Apache** y en **MySQL**.
3. Copia la carpeta `sistema_cafe` dentro de `C:\xampp\htdocs\`.
4. Abre en el navegador: `http://localhost/sistema_cafe/instalar.php` (una sola vez).
5. Abre el sistema: `http://localhost/sistema_cafe/`
6. Inicia sesión con el usuario **admin** y la clave **admin123**.

> **Importante:** los archivos `.php` no se abren con doble clic (eso abre el editor). Siempre deben abrirse por el navegador usando la dirección `http://localhost/...`, porque PHP necesita que Apache los procese.

> **Recorrido recomendado para aprender:** primero `config/database.php` (¿cómo se conecta?), luego `includes/functions.php` (los atajos), después `index.php` (login) y `auth.php` (seguridad), y por último un módulo completo como `proveedores.php`. Con eso se entiende el resto.

Anexo . Capturas de pantalla del sistema

Evidencia visual de cada modulo en funcionamiento, con datos de ejemplo.

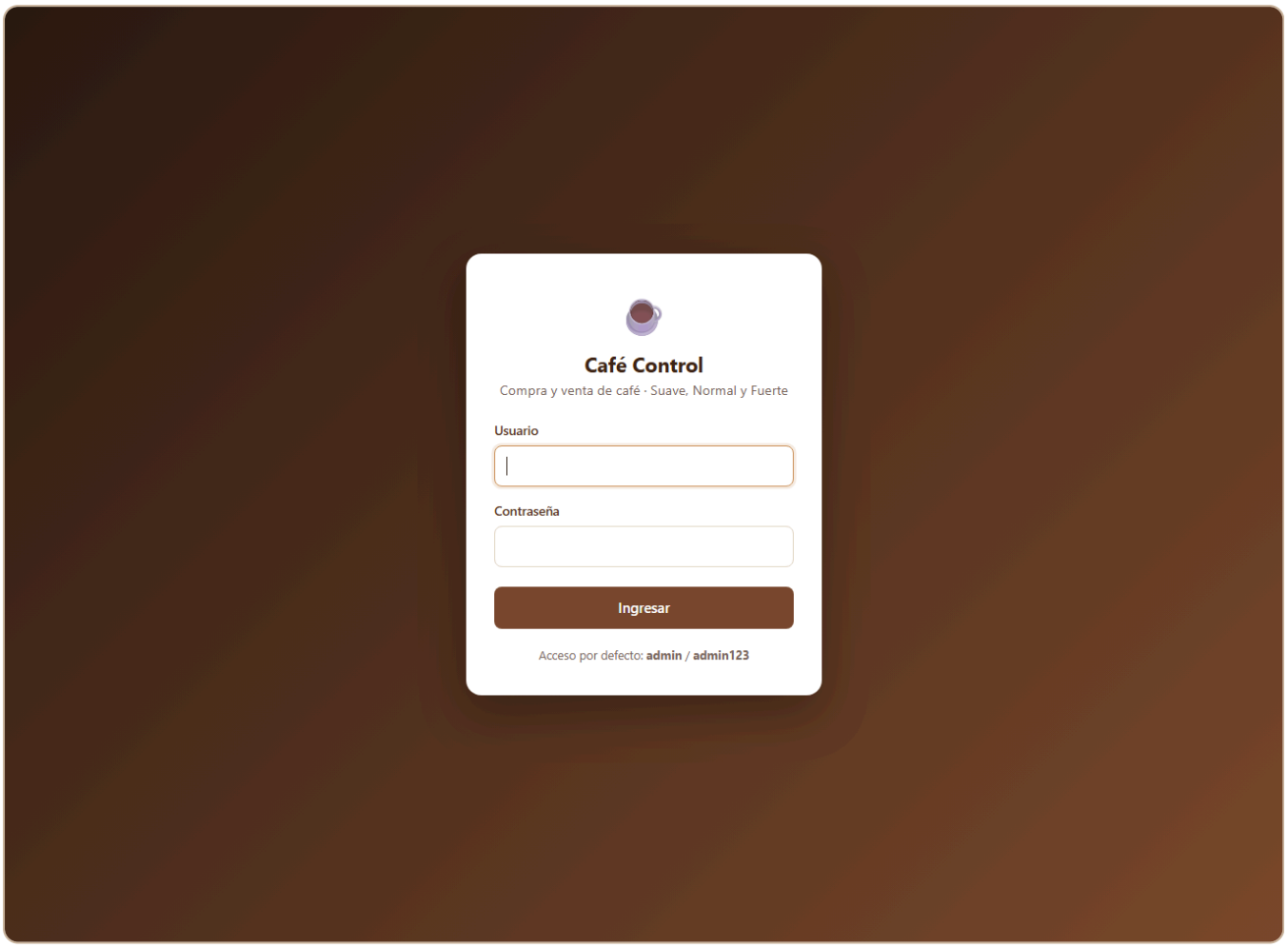


Figura 1. Seccion de acceso al sistema (login).

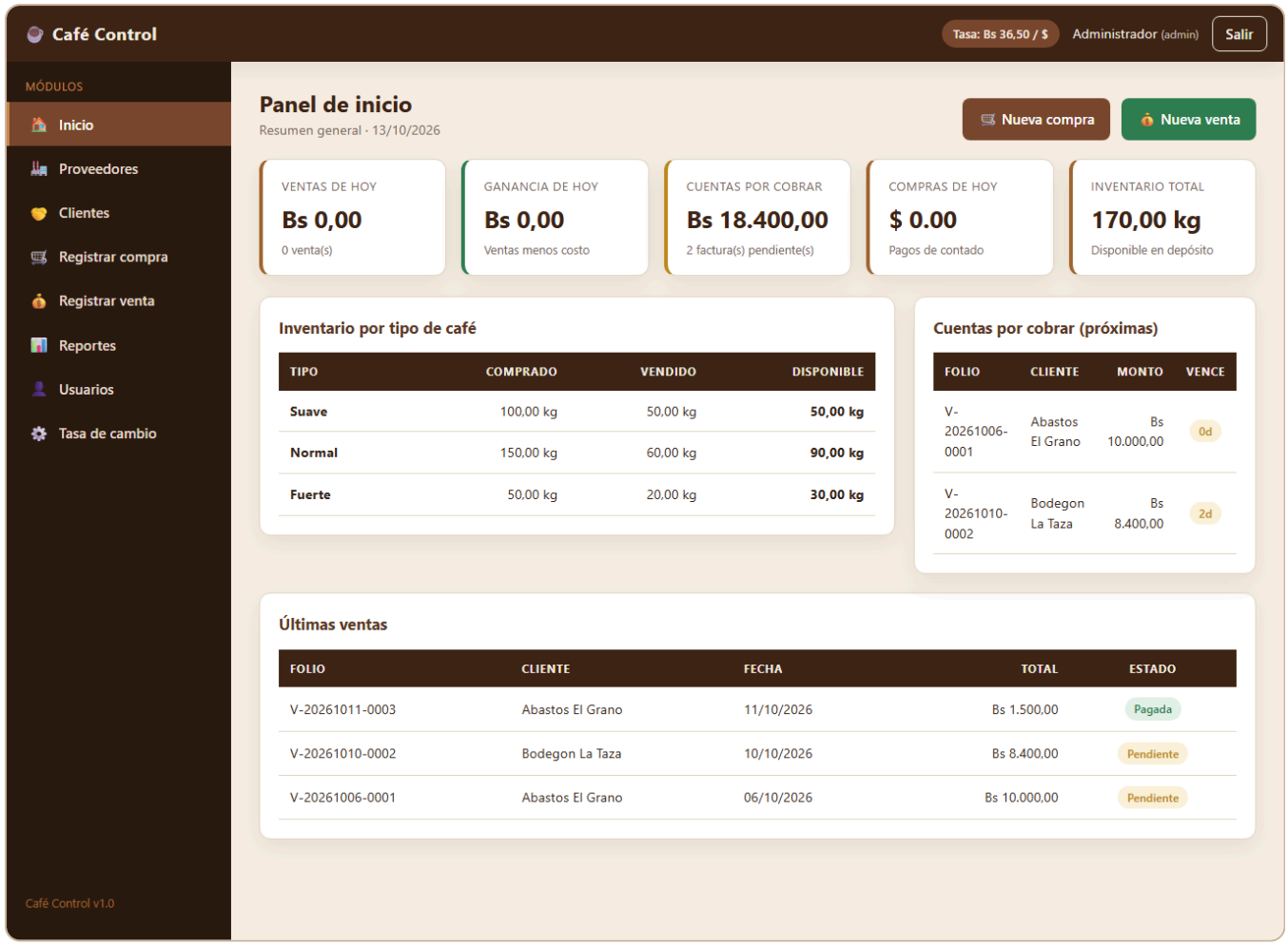


Figura 2. Panel de inicio con indicadores generales.

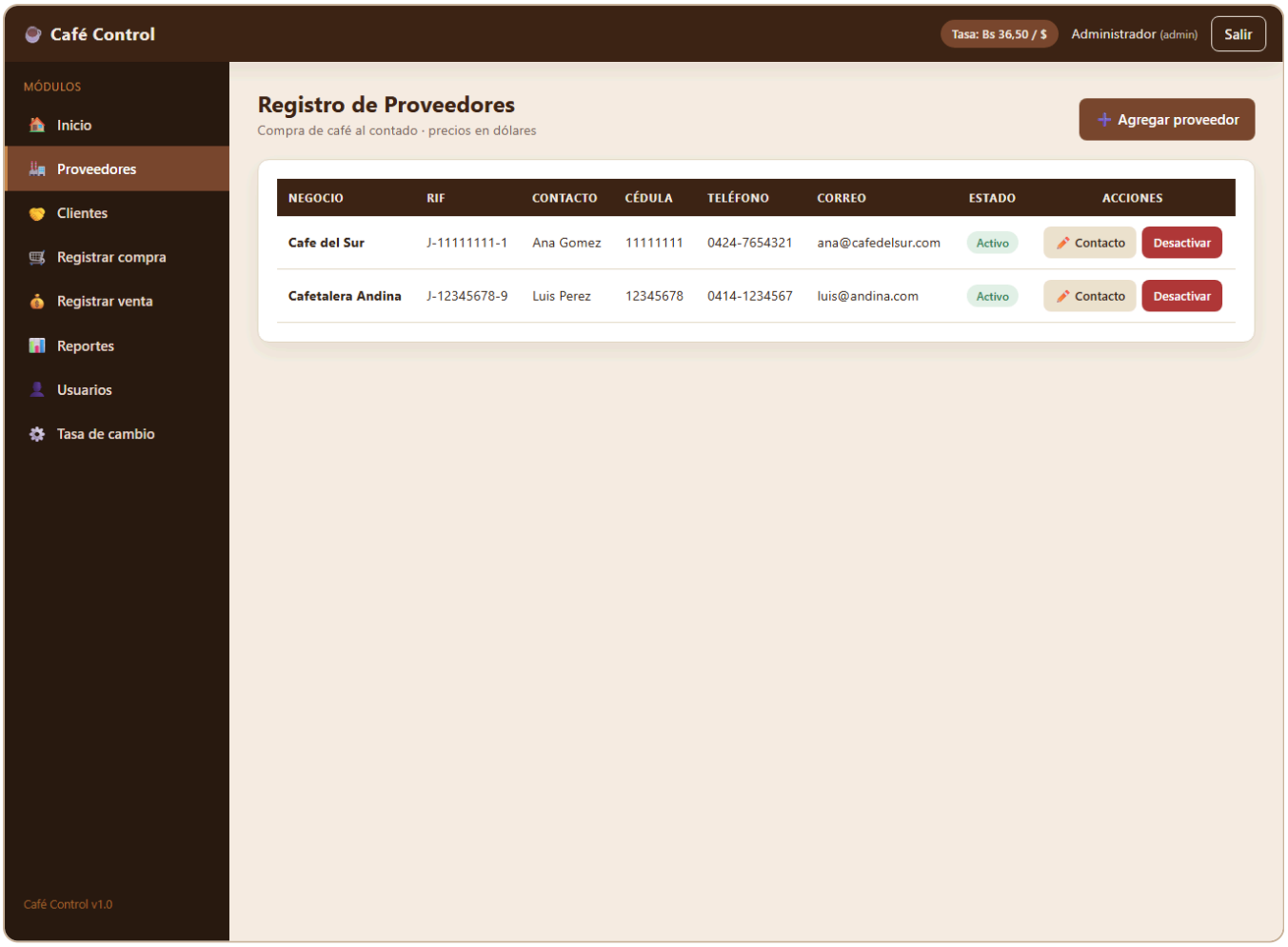


Figura 3. Modulo 1: Registro de Proveedores.

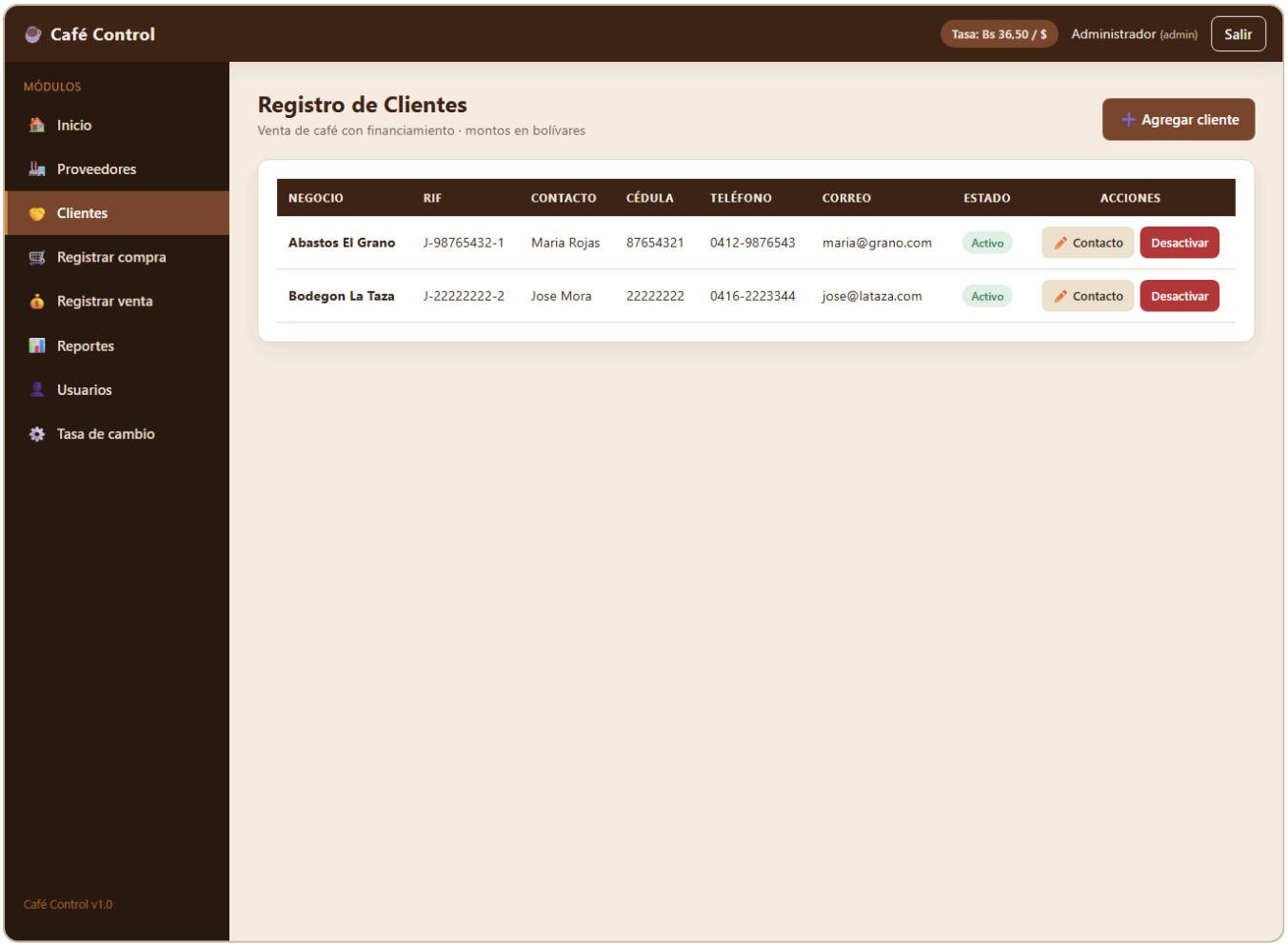


Figura 4. Modulo 2: Registro de Clientes.

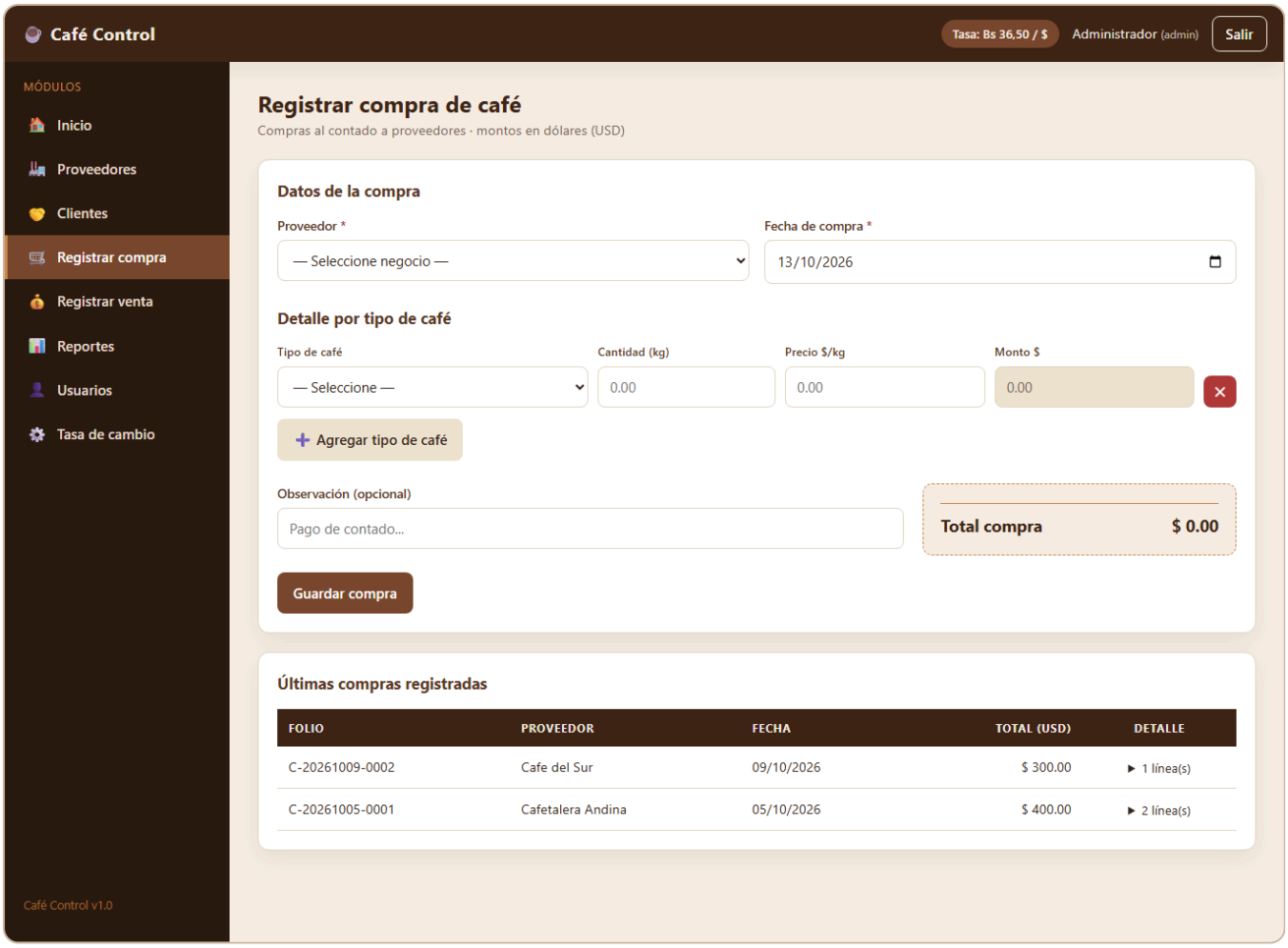


Figura 5. Modulo 3: Registrar compra de cafe (USD, al contado).

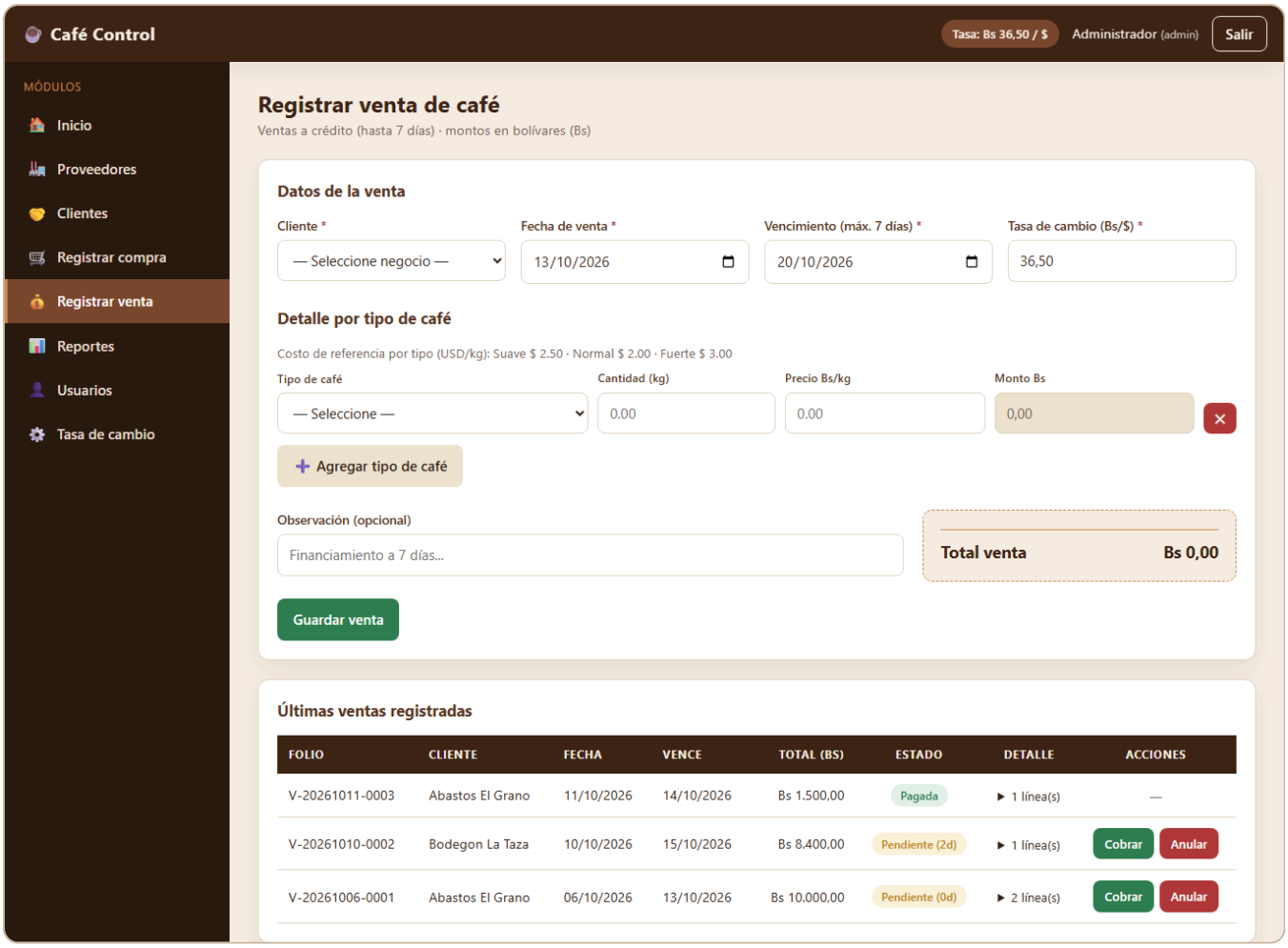


Figura 6. Modulo 3: Registrar venta de cafe (Bs, financiamiento hasta 7 dias).

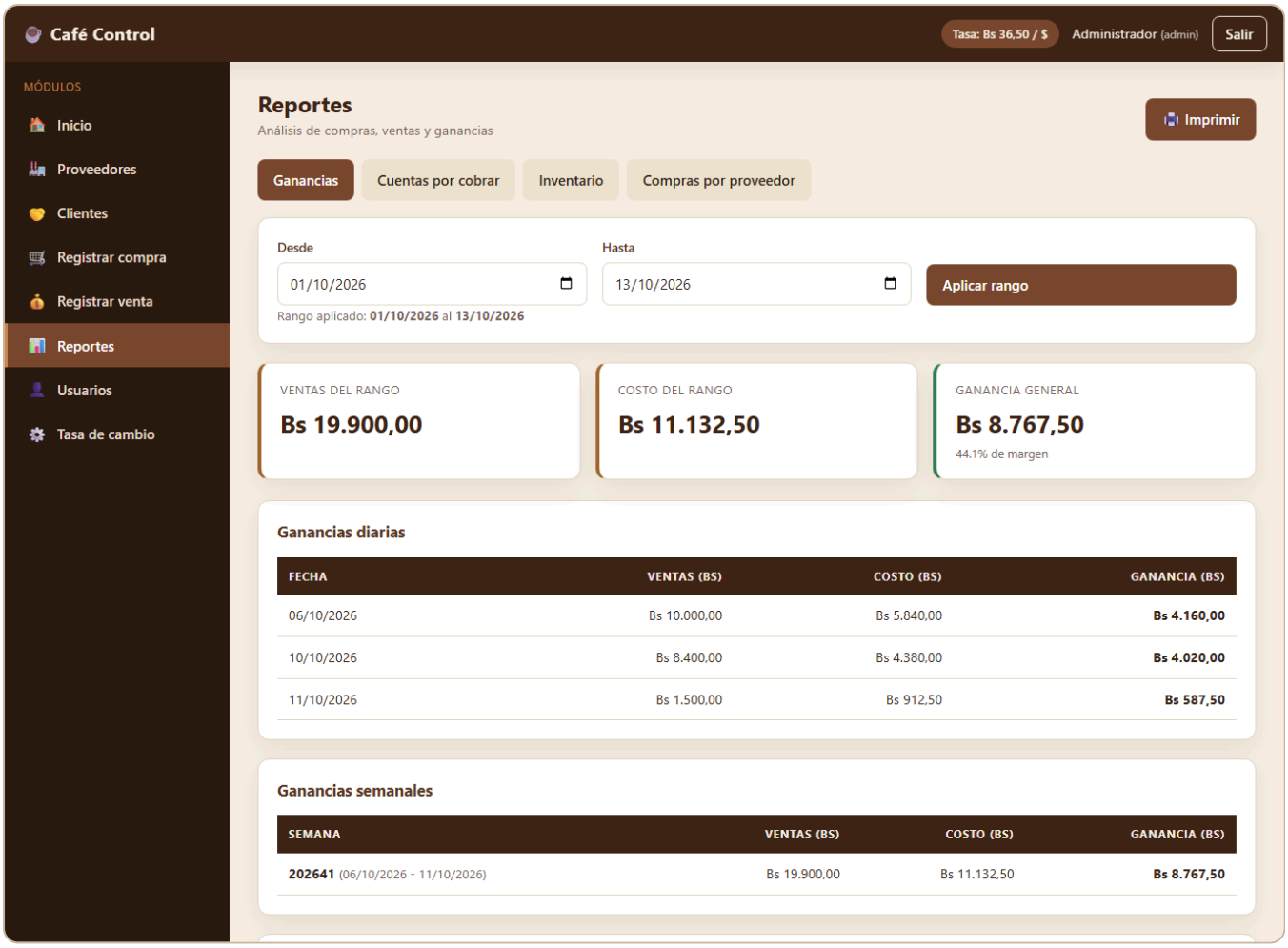


Figura 7. Modulo 4: Ganancias diarias, semanales y por negocio.

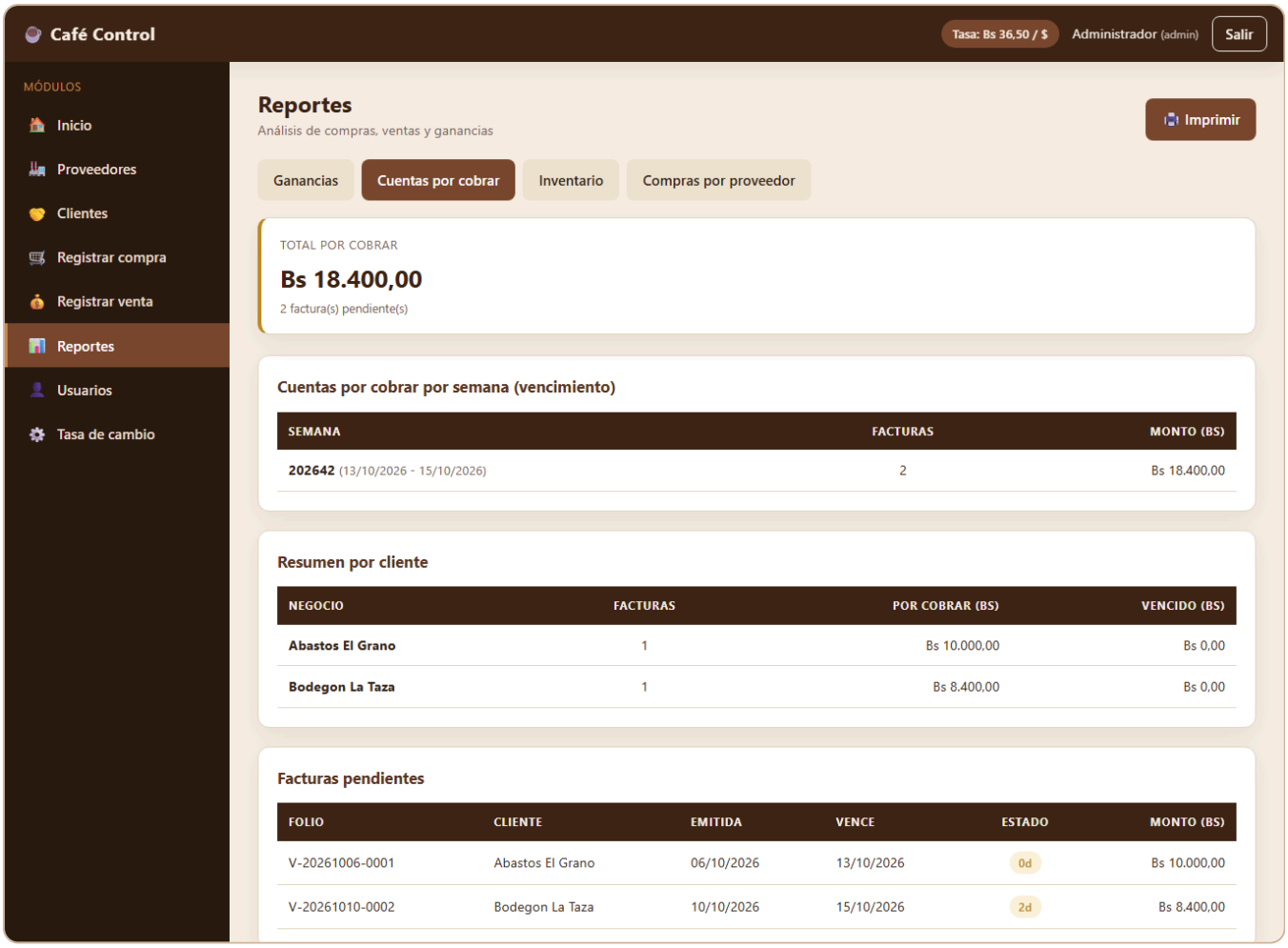


Figura 8. Modulo 4: Cuentas por cobrar semanal.

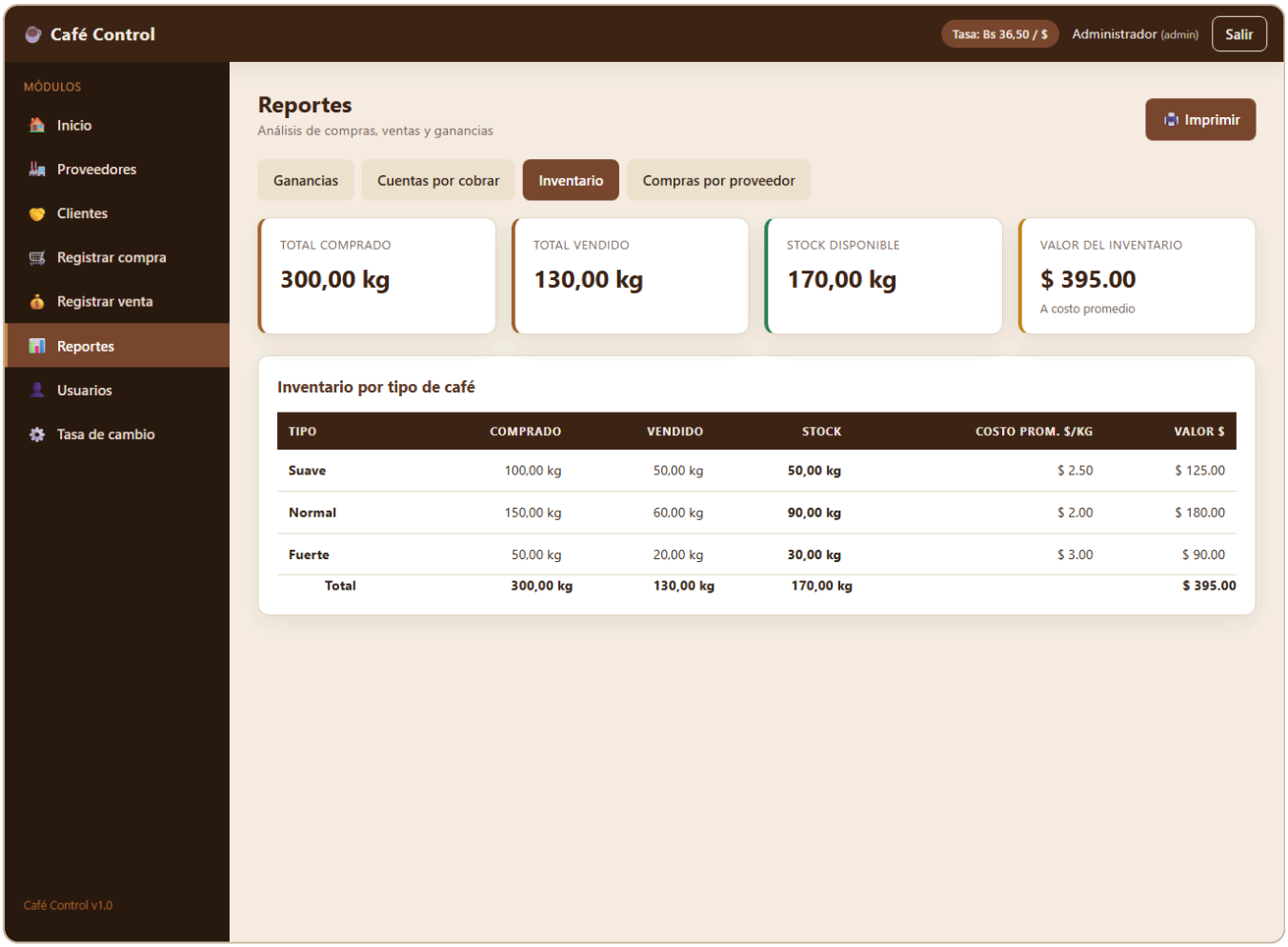


Figura 9. Reporte adicional: Inventario por tipo de cafe.

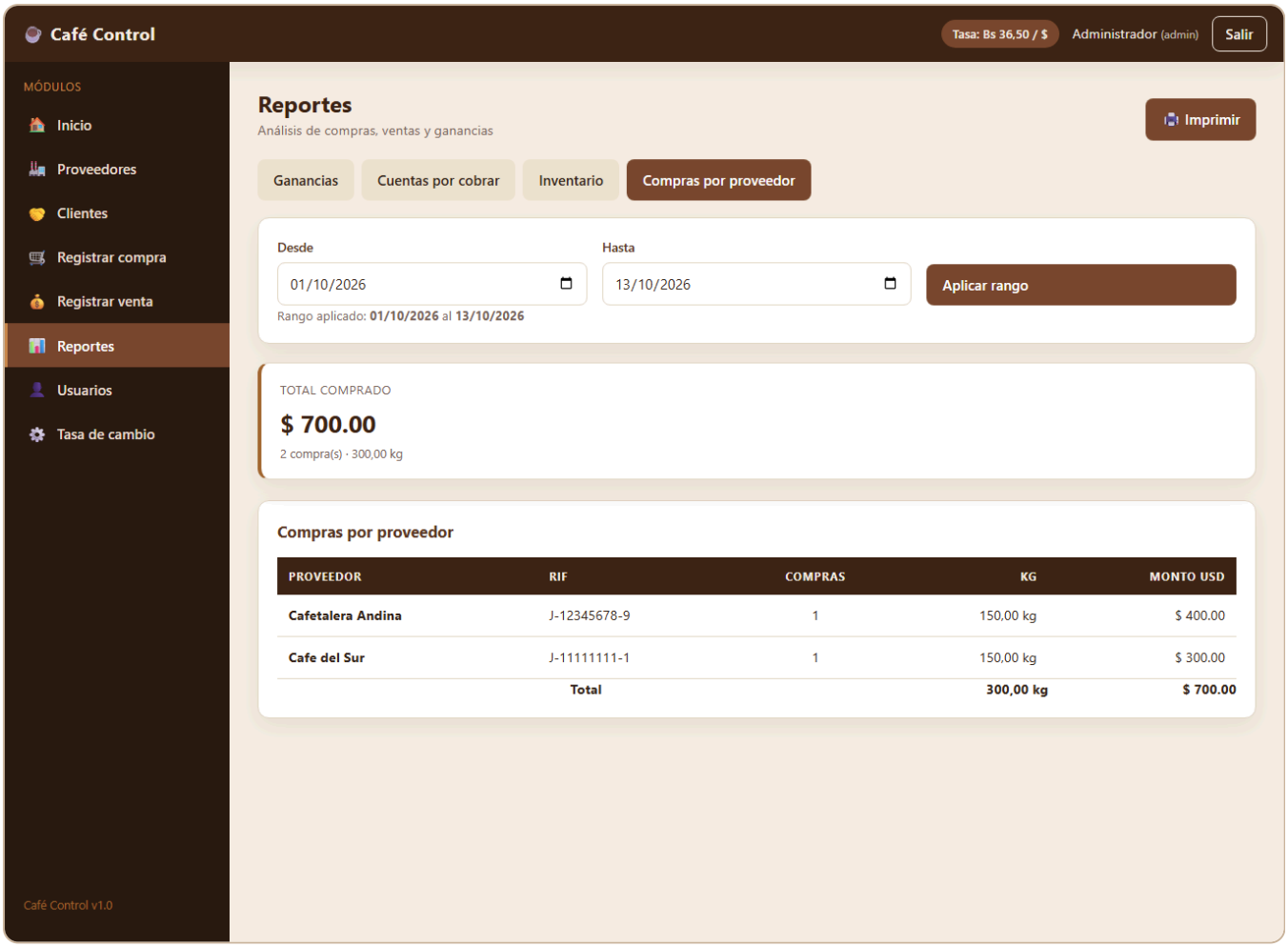


Figura 10. Reporte adicional: Compras por proveedor.

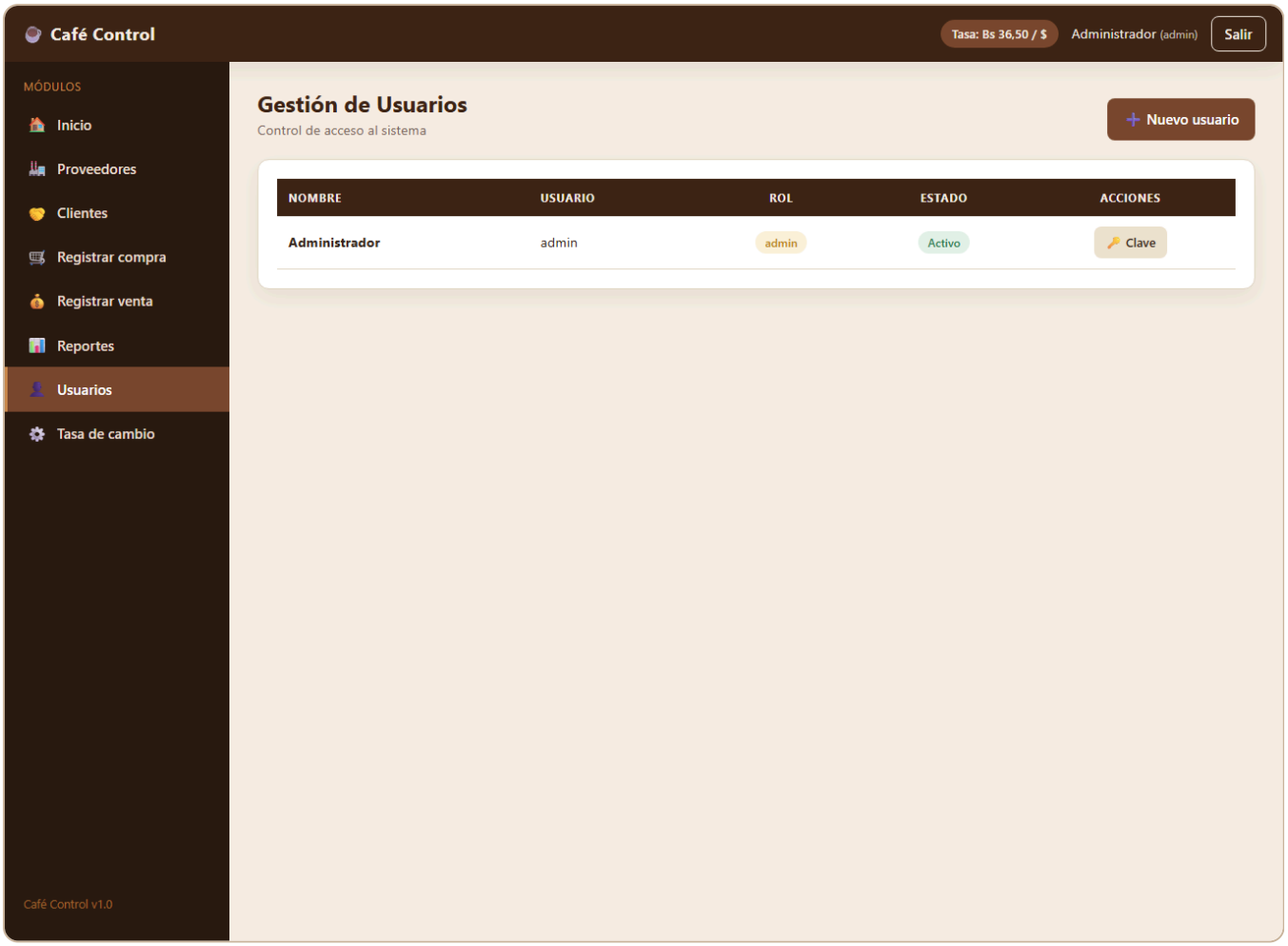


Figura 11. Gestion de Usuarios del sistema.

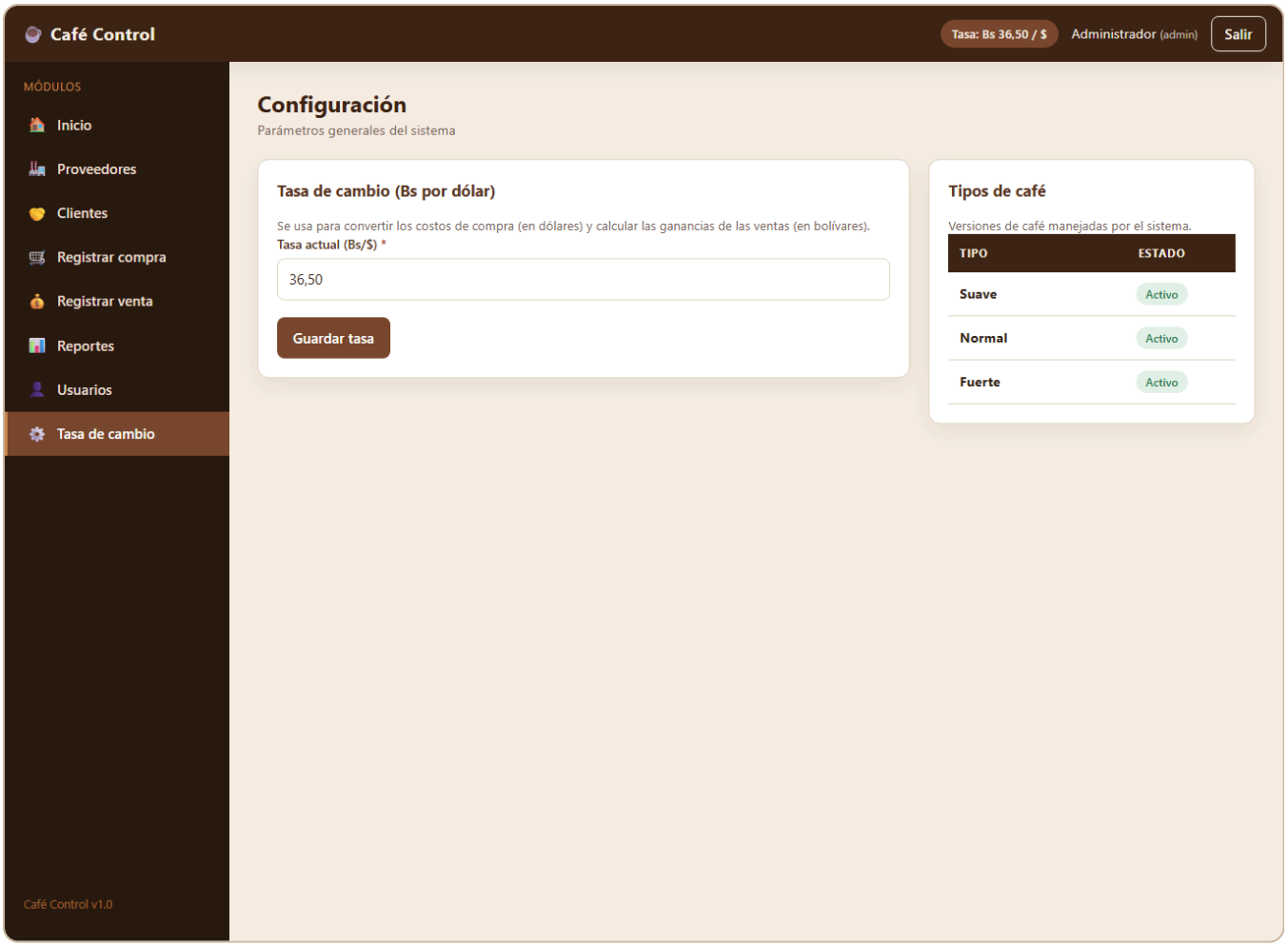


Figura 12. Configuracion de la tasa de cambio (Bs/$).